

PET DICOM Organizer

Program Documentation

Prepared by Ryan Byrd

October 16, 2025

1 Overview

PET DICOM Organizer is a Python utility designed to take a vendor-exported DICOM dataset (commonly distributed as a `DICOMDIR` accompanied by a nested folder structure of image files) and produce a clean, analysis-ready directory of standard `.dcm` files that is further organized into *Patient* $\rightarrow$ *Study* $\rightarrow$ *Series* hierarchy.

The tool:

- Reads a user-selected `DICOMDIR`.
- Resolves each `IMAGE` record to an actual file on disk using a multi-strategy resolver that tolerates missing extensions, mixed casing, shallow viewer-style folders (e.g., `A\Z04`), and nested DICOM exports.
- Copies already-valid DICOM files as `.dcm`; attempts GDCM-based conversion for non-DICOM encapsulations (`.IMA`, proprietary headers); and skips files that cannot be read.
- Emits a consolidated “Converted DICOMs” folder, then *re-organizes* those files into “Organized DICOMs/*Patient/Study/Series*” based on DICOM header tags.
- Cleans up intermediate temporary output to leave only the final, organized dataset.

This workflow streamlines downstream analysis (e.g., uniformity assessment, SUV processing, or archiving) by normalizing file extensions, sanitizing names, and imposing a stable directory layout.

2 Program Workflow

2.1 Step 1: Input Selection

On launch, the program presents three dialogs:

1.1 Choose DICOMDIR: The master directory record file exported by the scanner or viewer.

1.2 Choose Image Root: The top-level folder that contains all exported images (e.g., `D:\DICOM`).

1.3 Choose Output Folder: A destination where the tool writes temporary and final results.

If any selection is canceled, the program exits with a message indicating as such.

2.2 Step 2: Recursive Indexing of the Image Root

The image root is scanned recursively to build two complementary indices:

- **by leaf:** Maps each filename *stem* (basename minus extension) to all full paths that share that stem.

- **by_rel:** Maps each file's *relative path* (with and without extension, using backslashes) to its absolute path.

These indices allow robust lookups no matter whether the DICOMDIR refers to images by a relative path, a partial path, or a bare leaf name.

2.3 Step 3: Parsing DICOMDIR

The program reads the DICOMDIR and extracts all entries whose `DirectoryRecordType` is `IMAGE`. For each `IMAGE` record, the referenced path (tag `ReferencedFileID`) may be a sequence of components (e.g., "50140002", "18507241"), which the program flattens into a normalized relative path.

2.4 Step 4: Reference Resolution Strategy

For each `IMAGE` reference, the resolver attempts, in order:

- a) **Direct join:** Concatenate image root with the reference; try with common extensions (`.ima`, `.IMA`, `.dcm`, `.DCM`) and with/without extension.
- b) **Index lookup by relative path:** Query `by_rel` using the normalized path (with and without extension).
- c) **Index lookup by leaf:** Use the basename stem to retrieve all candidate paths, then pick the best match by:
 - Preferring the shortest matching relative path; and
 - Applying a simple score that rewards overlap between the requested path's directory parts and the candidate path.

If no file can be resolved, the reference is recorded as *missing*.

2.5 Step 5: DICOM Validation and Conversion

Once a source file is found:

- **Validation:** A permissive *pydicom* read ("force" mode, stop before pixel data) determines if the file is a probable DICOM, even if it lacks the DICM preamble.
- **Copy vs. Convert:**
 - If valid, the file is copied verbatim to the *Converted_DICOMs* folder with a `.dcm` extension.
 - Otherwise, the program attempts a GDCM decode/encode cycle. If successful, a standard `.dcm` is written; if not, the file is counted as *failed*.

A running summary keeps counts of successfully copied, successfully converted, and missing/failed items.

2.6 Step 6: Organization into Patient/Study/Series

All files in *Converted_DICOMs* are then re-parsed (header-only) and copied into a stable hierarchy:

Organized_DICOMs/

PatientName_PatientID /
sanitized
StudyDate_StudyDescription /
sanitized
SeriesNumber_SeriesDescription
sanitized

File- and folder-names are sanitized to contain only alphanumerics, spaces, underscores, and hyphens.

2.7 Step 7: Cleanup of Intermediates

To leave a clean deliverable for analysis and archival, the temporary *Converted_DICOMs* directory is deleted. The final output is *Organized_DICOMs* inside the user-selected output folder.

3 Interpretation of Results

The program concludes with a concise terminal summary:

- **Copied clean DICOMs:** Count of files that were already valid and simply re-suffixed to .dcm.
- **GDCM-converted:** Count of files that required and survived conversion.
- **Missing/failed:** IMAGE references that could not be resolved or converted.
- **Final location:** Path to the *Organized_DICOMs* directory ready for downstream use.

A high *missing/failed* rate typically indicates that the chosen image root does not correspond to the DICOMDIR provenance, or that the export is incomplete.

4 How to Use the Program

1. **Launch the script** (`PET_DICOM_Organizer.py`) in a Python environment with the dependencies below.
2. **Select the DICOMDIR** file when prompted.
3. **Select the image root** (top-level directory that contains all referenced images).
4. **Select the output folder** (a working directory where the organizer can write results).
5. **Review the summary** in the terminal when processing completes.
6. Use `Organized_DICOMs/` for analysis (import into viewers, run QA pipelines, etc.).

5 Notes and Recommendations

5.1 Dependencies

- **Python** 3.8+ (tested variants may differ by site).
- **Packages:** `pydicom`, `gdcm` (`python-gdcm`), `tqdm`.
- **UI:** Built-in `tkinter` for native file/folder selection dialogs.

Ensure `python-gdcm` is properly installed for your platform; on Windows this often requires a wheel matching your Python version.

5.2 Supported Inputs

- Image records referenced as multi-part paths in DICOMDIR (with or without extensions).
- Mixed file extensions and casings (`.IMA`, `.ima`, `.DCM`, `.dcm`) and files lacking extensions entirely.
- Viewer-style shallow exports (e.g., `A\Z04`) as well as deeply nested DICOM backups.

5.3 Common Failure Modes

- **Incorrect image root:** If the selected image root does not actually contain the referenced files, resolution fails.
- **Partial export:** Some modalities export only thumbnails or omit certain series; affected IMAGE records will be missing.
- **Non-decodable encapsulation:** Rarely, GDCM cannot decode proprietary formats; these will be logged as failed.

5.4 Performance Considerations

- The recursive index is built once and reused, which amortizes lookup cost across many IMAGE records.
- The resolver’s tiered strategy (direct join $\rightarrow$ index by relative path $\rightarrow$ index by leaf) minimizes disk hits.
- Progress bars (`tqdm`) show throughput; very large studies with deep trees may take several minutes.

5.5 Naming and Sanitization

To ensure cross-platform compatibility and avoid shell/Excel parsing pitfalls, patient, study, and series names are “sanitized” to alphanumeric, spaces, underscores, and hyphens only. Original DICOM `PatientName`, `StudyDescription`, and `SeriesDescription` remain intact in the headers for downstream use.

6 Conclusion

PET DICOM Organizer provides a robust, vendor-agnostic pathway from heterogeneous exports to a clean, standardized, and analysis-ready DICOM directory structure. By combining tolerant reference resolution, best-effort GDCM conversion, and structured reorganization, it

reduces manual curation time and improves reproducibility for physics QA, research processing, and clinical audit workflows.